

Assignment No 2: Feedforward Neural Network Implementation with Keras and TensorFlow

- Import the necessary packages
- Load the training and testing data (MNIST/CIFAR10)
- Define the network architecture using Keras
- Train the model using SGD
- Evaluate the network
- Plot the training loss and accuracy

```
In [ ]: # !pip install tensorflow matplotlib
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
import random
import numpy as np
```

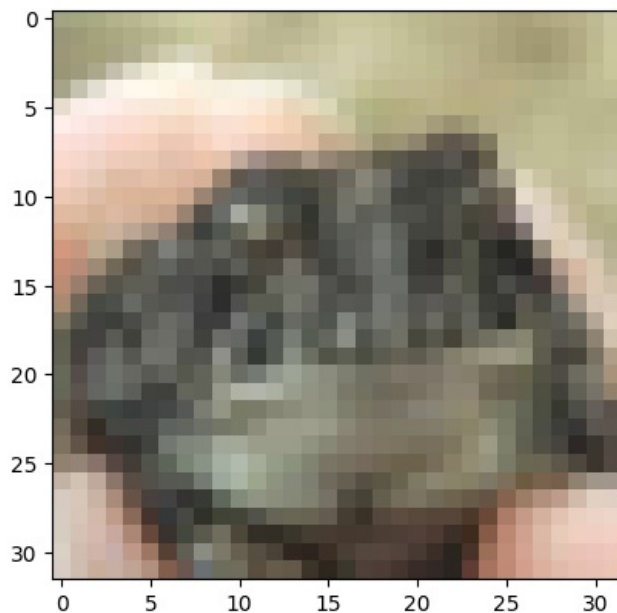
```
In [ ]: cifar10 = tf.keras.datasets.cifar10 # Importing cifar10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data() # Splitting it into training and testing data

for i in (x_train, y_train, x_test, y_test):
    print(i.shape)
```

```
(50000, 32, 32, 3)
(50000, 1)
(10000, 32, 32, 3)
(10000, 1)
```

```
In [ ]: plt.imshow(x_train[25])
print(y_train[25])
plt.show()
```

```
[6]
```



```
In [ ]: #scaling the values
x_train = x_train/255
x_test = x_test/255
# 0/255 = 255
#255/255 = 1 maximum value
```

```
In [ ]: print(x_train.shape, x_test.shape)

(50000, 32, 32, 3) (10000, 32, 32, 3)
```

```
In [ ]: num_classes = 10
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
```

```
In [ ]: # Convert class vectors to binary class matrices.
y_train = tf.keras.utils.to_categorical(y_train, num_classes)
y_test = tf.keras.utils.to_categorical(y_test, num_classes)
```

```
In [ ]: print(y_train.shape, y_test.shape)
```

(50000, 10) (10000, 10)

```
In [ ]: from tensorflow import keras

# Create a sequential model
model = keras.Sequential()

# Convolutional layers
model.add(keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(keras.layers.MaxPooling2D((2, 2)))
model.add(keras.layers.Conv2D(64, (3, 3), activation='relu'))
model.add(keras.layers.MaxPooling2D((2, 2)))
model.add(keras.layers.Conv2D(64, (3, 3), activation='relu'))

# Fully connected layers
model.add(keras.layers.Flatten())
model.add(keras.layers.Dense(64, activation='relu'))
model.add(keras.layers.Dense(10, activation='softmax')) # 10 classes for classification

# Compile the model
model.compile(optimizer='sgd', loss='categorical_crossentropy', metrics=['accuracy'])

# Print the model summary to check the architecture
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36928
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 64)	65600
dense_1 (Dense)	(None, 10)	650
=====		
Total params: 122570 (478.79 KB)		
Trainable params: 122570 (478.79 KB)		
Non-trainable params: 0 (0.00 Byte)		

```
In [ ]: # Training the model
history = model.fit(x_train, y_train,
                    batch_size=30,
                    epochs=10,
                    validation_data=(x_test, y_test),
                    shuffle=True)
```

```

Epoch 1/10
1667/1667 [=====] - 21s 9ms/step - loss: 2.0769 - accuracy: 0.2370 - val_loss: 1.7587 -
val_accuracy: 0.3696
Epoch 2/10
1667/1667 [=====] - 10s 6ms/step - loss: 1.6585 - accuracy: 0.4016 - val_loss: 1.8162 -
val_accuracy: 0.3632
Epoch 3/10
1667/1667 [=====] - 10s 6ms/step - loss: 1.4876 - accuracy: 0.4663 - val_loss: 1.4178 -
val_accuracy: 0.4869
Epoch 4/10
1667/1667 [=====] - 9s 6ms/step - loss: 1.3671 - accuracy: 0.5106 - val_loss: 1.3243 -
val_accuracy: 0.5263
Epoch 5/10
1667/1667 [=====] - 9s 5ms/step - loss: 1.2771 - accuracy: 0.5460 - val_loss: 1.2605 -
val_accuracy: 0.5553
Epoch 6/10
1667/1667 [=====] - 10s 6ms/step - loss: 1.1983 - accuracy: 0.5791 - val_loss: 1.1841 -
val_accuracy: 0.5851
Epoch 7/10
1667/1667 [=====] - 11s 7ms/step - loss: 1.1324 - accuracy: 0.6015 - val_loss: 1.1422 -
val_accuracy: 0.5981
Epoch 8/10
1667/1667 [=====] - 11s 7ms/step - loss: 1.0736 - accuracy: 0.6219 - val_loss: 1.1310 -
val_accuracy: 0.6059
Epoch 9/10
1667/1667 [=====] - 9s 5ms/step - loss: 1.0227 - accuracy: 0.6426 - val_loss: 1.0342 -
val_accuracy: 0.6357
Epoch 10/10
1667/1667 [=====] - 10s 6ms/step - loss: 0.9766 - accuracy: 0.6567 - val_loss: 1.0536 -
val_accuracy: 0.6301

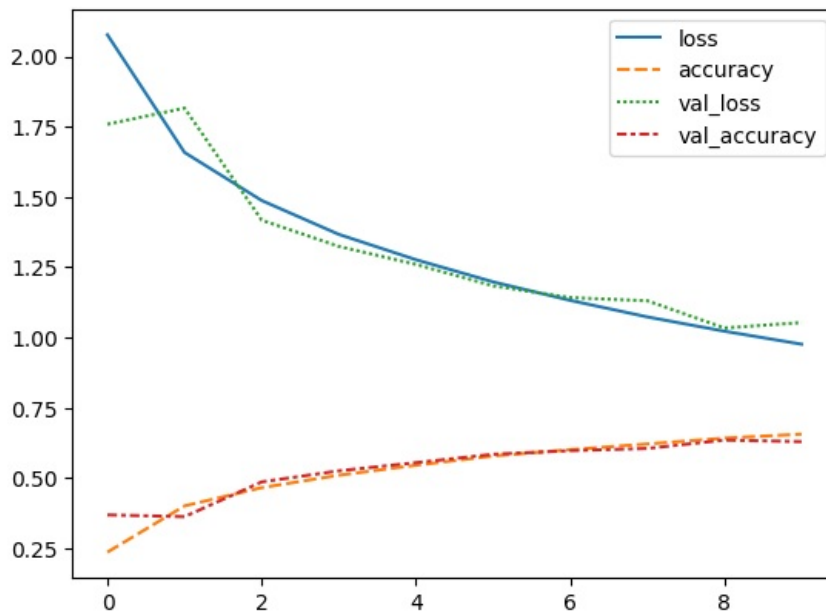
```

```

In [ ]: import seaborn as sns
sns.lineplot(model.history.history)

```

Out[]: <Axes: >



Alternative Plot below for more details

```

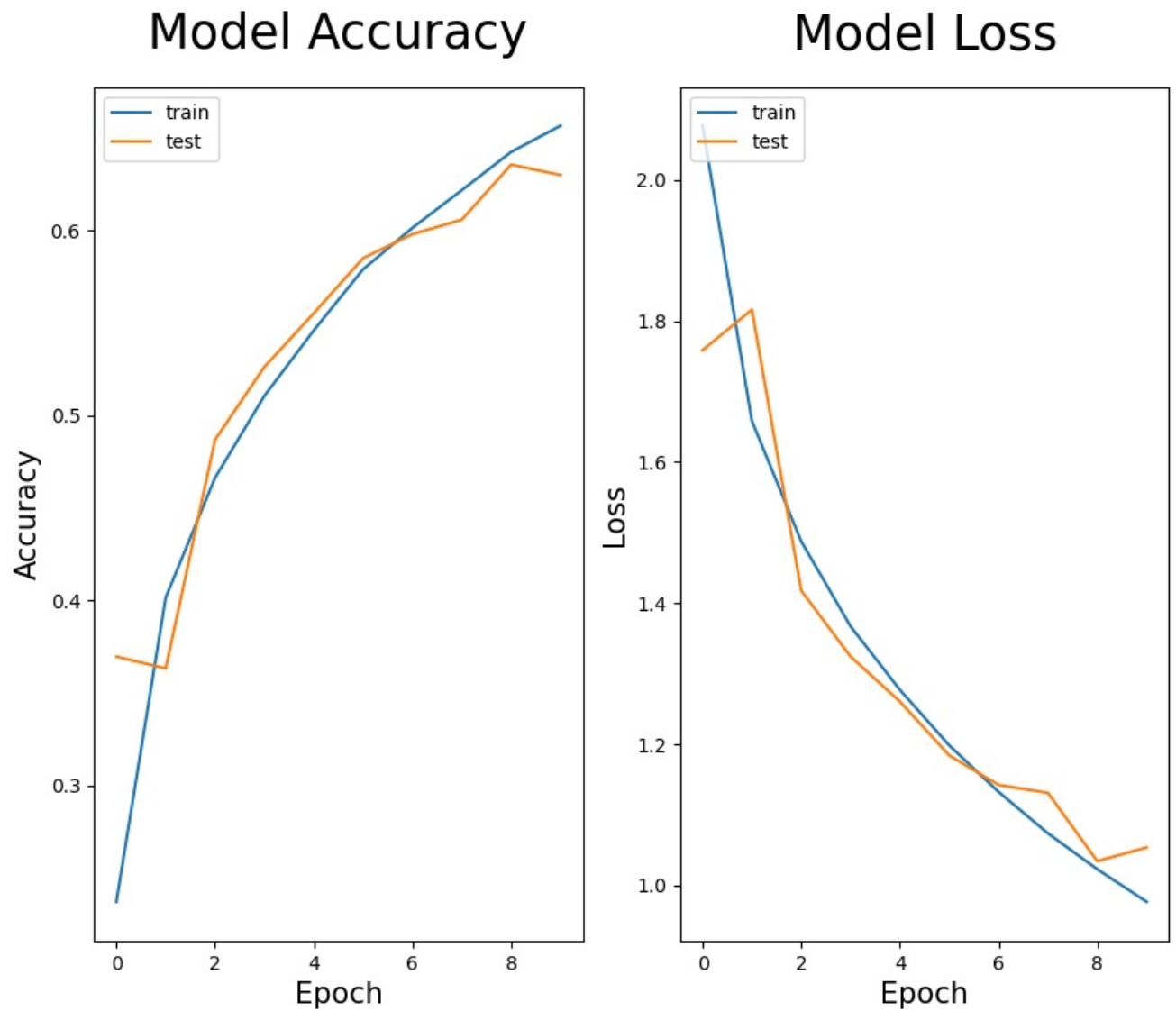
In [ ]: # Plotting the Model Accuracy & Model Loss vs Epochs
plt.figure(figsize=[10,8])

# summarize history for accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy', size=25, pad=20)
plt.ylabel('Accuracy', size=15)
plt.xlabel('Epoch', size=15)
plt.legend(['train', 'test'], loc='upper left')

# summarize history for loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss', size=25, pad=20)
plt.ylabel('Loss', size=15)
plt.xlabel('Epoch', size=15)

```

```
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```



Evaluation

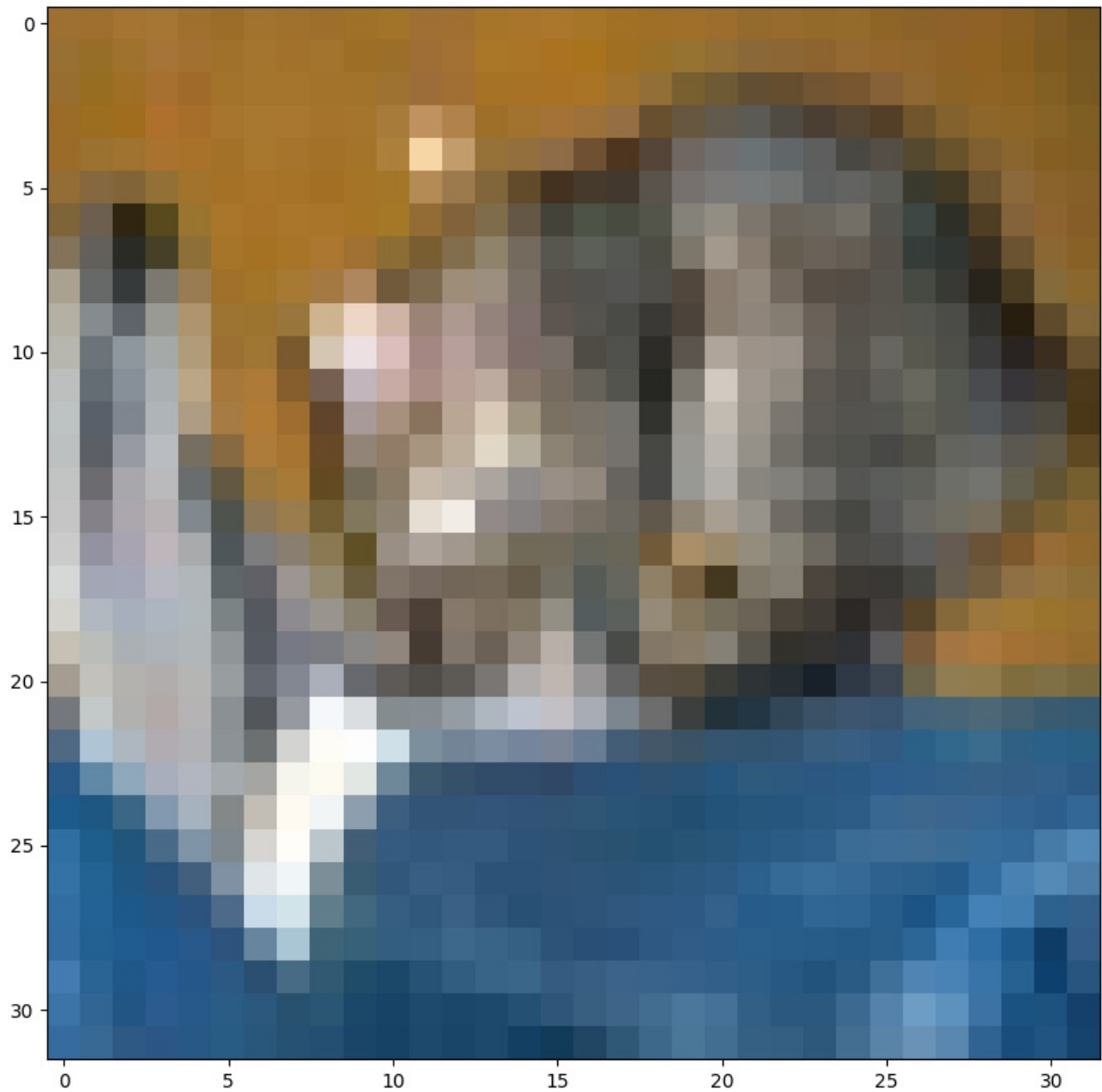
```
In [ ]: predictions = model.predict(x_test)
```

```
313/313 [=====] - 1s 2ms/step
```

```
In [ ]: class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
```

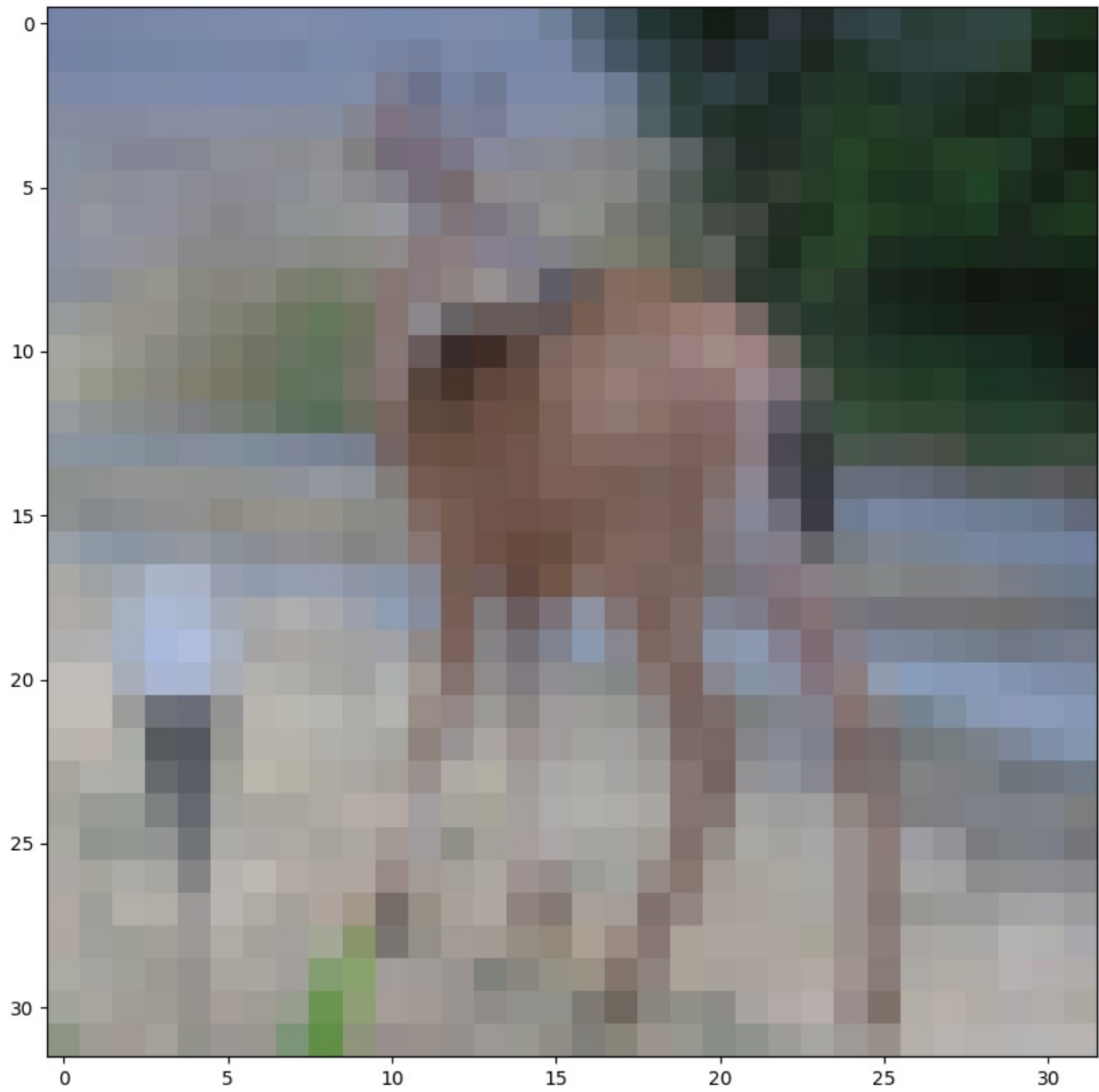
```
In [ ]: plt.figure(figsize=[10,10])
plt.imshow(x_test[0].reshape(32, 32, -1))
plt.title("Predicted value: " + str(class_names[np.argmax(predictions[0], axis=0)]), size=20)
plt.grid(False)
```

Predicted value: cat



```
In [ ]: plt.figure(figsize=[10,10])
plt.imshow(x_test[100].reshape(32, 32, -1))
plt.title("Predicted value: " + str(class_names[np.argmax(predictions[100], axis=0)]), size=20)
plt.grid(False)
```

Predicted value: deer



In []:

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js